SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST



TUGBOAT

A self-hosted, push-to-deploy platform built on Buildpacks, Docker, Traefik and Cloudflare Tunnel

Detailed Project Report

Submitted by

Pradyum Mistry - 16010123236

B.Tech Computer Engineering, Last Year

K. J. Somaiya School of Engineering, Mumbai

DevOps Mini Project

October 2026

Repository: github.com/altf4-games/tugboat

Abstract

Putting a web application online usually means writing a Dockerfile, running the container, configuring a reverse proxy and TLS, and exposing the result to the internet. Hosted platforms such as Vercel, Railway and Render hide these steps, but they are metered and closed, so a student or a small team cannot see how they work.

Tugboat is a self-hosted platform that does the same job on one machine using open-source tools. A user signs in with GitHub and links a repository, and Tugboat adds a webhook to it. Every push is cloned and built into an image with Cloud Native Buildpacks, so no Dockerfile is needed. The image runs as a Docker container with labels that Traefik reads to route a hostname to it, and a Cloudflare quick tunnel gives it a public HTTPS address. A push to the default branch also promotes the image to production with a blue-green swap that waits for a health check before any traffic moves. A rollback re-promotes the previous image. A web dashboard shows deployments, streams build logs, and manages environment variables, memory and CPU limits, and webhook history.

I built the system test-first and ran the tests against the real tools: Docker, pack, Traefik and cloudflared. The last complete run had 74 tests in 28 files. One integration test sends a request every 50 ms while production is promoted twice in a row, and none of the requests fail. This report describes the design, the deployment pipeline, the tests, the problems I found along the way, and the limits of the current implementation.

Keywords: *DevOps, platform as a service, Cloud Native Buildpacks, Docker, Traefik, Cloudflare Tunnel, blue-green deployment, webhooks.*

Table of contents

1. Introduction....6
1.1 Background....6
1.2 Problem statement....6
1.3 Objectives....6
1.4 Scope....7
2. Background and related concepts....8
2.1 Push-to-deploy platforms....8
2.2 Cloud Native Buildpacks....8
2.3 Containers and Docker....8
2.4 Reverse proxies and service discovery....8
2.5 Tunnels....9
2.6 Blue-green deployment and rollback....9
2.7 Webhooks and request signing....9
3. Technology stack....10
3.1 Components....10
3.2 Dependencies....11
4. System architecture....12
4.1 Overview....12
4.2 Control plane modules....13
4.3 Networking and ports....14
4.4 Repository structure....14
5. Application design....15
5.1 Features....15
5.2 REST API....15
5.3 Data model....16
5.4 Authentication and sessions....17
5.5 The web interface....17
6. Deployment pipeline....18
6.1 Push: webhook intake....18
6.2 Build....18
6.3 Run and route....19
6.4 Share: tunnel, DNS and screenshot....19
6.5 Promote: blue-green swap....20
6.6 Rollback....21
6.7 Branch deletion and unlinking....21
6.8 Concurrency control....21
7. Containerisation and runtime controls....23
7.1 Image build....23
7.2 Container configuration....23

7.3 Traefik and the Docker API proxy....24
7.4 Resource limits....24
8. Testing....25
8.1 Approach....25
8.2 Test suite....25
8.3 Results....25
9. Environment configuration....27
9.1 Prerequisites....27
9.2 Configuration....27
9.3 Running and cleaning up....28
10. End-to-end workflow....29
11. Problems encountered and solutions....31
12. Conclusion and future work....32
12.1 Conclusion....32
12.2 Limitations....32
12.3 Future work....32
References....33

List of figures

Figure 1: Technology stack by layer....10
Figure 2: High-level system architecture....12
Figure 3: Database tables....16
Figure 4: Landing page and project page (the project page is shown with sample data)....17
Figure 5: Deployment pipeline for one push....18
Figure 6: Blue-green promotion of a new image....20
Figure 7: End-to-end sequence for one push....29

List of tables

Table 1: Technology components and roles....10
Table 2: Node.js dependencies of the control plane....11
Table 3: Control plane modules....13
Table 4: Ports and networks....14
Table 5: Route summary....15
Table 6: What happens when a stage fails....22
Table 7: How each container is configured....23
Table 8: Traefik start-up flags....24
Table 9: Test suite overview....25
Table 10: Prerequisites....27
Table 11: Configuration variables....27
Table 12: Problems and solutions....31

1. Introduction

1.1 Background

Continuous integration and delivery shortened the time between writing code and running it. Platforms such as Heroku, and later Vercel, Railway and Render, went further and reduced the workflow to three actions: connect a repository, push, and open a URL. Behind that interface the platform does a fixed set of jobs. It builds the code into something runnable, runs it in isolation, routes a hostname to it, terminates TLS, and replaces the running version without interrupting users.

Each of those jobs has a mature open-source tool. Cloud Native Buildpacks build images without a Dockerfile, Docker runs them, Traefik routes to containers as they appear, and Cloudflare Tunnel exposes a local port over HTTPS without opening any ports. What is missing is the glue that connects them to a Git push. I wrote that glue myself to see what a deployment platform does.

1.2 Problem statement

Taking a project from a Git push to a reachable URL on a developer's own machine takes many manual steps, and each step needs its own tool and configuration. Three gaps stand out:

1. Every application needs a Dockerfile that someone writes and maintains.
2. Routing needs a reverse proxy that is configured, and reloaded when containers start and stop.
3. Sharing a running version with another person needs open ports, DNS and a certificate.

Hosted platforms close these gaps, but they are metered, they run on someone else's infrastructure, and their internals are not visible. This project addresses how to give a single developer the same push-to-deploy workflow on their own machine, using open-source parts, in a way that can be tested for real.

1.3 Objectives

1. Receive and verify GitHub push and branch-delete events for any repository the user links.
2. Build an image from the pushed commit without requiring a Dockerfile.
3. Run every branch as its own container and route to it automatically, with no proxy configuration edits and no restarts.
4. Give every deployment a public HTTPS link without port forwarding or DNS setup.
5. Promote the default branch to a production route without dropping requests, and allow a rollback to the previous image.

6. Provide a web dashboard with live build logs, per-project settings, and a complete clean-up when a project is unlinked.
7. Test every feature against the real tools, with no mocked services and no seeded data.

1.4 Scope

Tugboat runs on one machine and is meant for one developer, or a small group that trusts each other. State is kept in a local SQLite file. The applications it deploys are ones that Cloud Native Buildpacks can detect, and they must listen on the port given in the `PORT` environment variable. To be promoted to production an application must also answer `GET /health`. There is one production route, shared by every project. Quick tunnel addresses are temporary and change with each deployment.

Multi-node scheduling, custom domains, private registries, encrypted secret storage, per-user access control and billing are outside the scope. Chapter 12 lists the main ones as future work.

2. Background and related concepts

2.1 Push-to-deploy platforms

A push-to-deploy platform watches a Git repository and deploys each new commit automatically. Heroku popularised the workflow. The Twelve-Factor App [1] describes the application conventions that make it work: configuration is read from environment variables, the app binds to a port given by the platform, and the process keeps no state of its own. Tugboat follows the same conventions. It passes `PORT` and the project's environment variables to the container and expects the application to read them.

2.2 Cloud Native Buildpacks

Buildpacks turn source code into a container image without a Dockerfile. The Cloud Native Buildpacks specification [2] splits the work into two phases. In detection, each buildpack in a builder checks whether it applies to the source, for example by finding a `package.json`. In build, the matching buildpacks install the runtime and the dependencies and add them to the image as layers. A builder bundles the buildpacks with a build image and a run image. Tugboat uses the Paketo builder `paketobuildpacks/builder-jammy-base` [3] through the `pack` command line tool. Layers are cached between builds, so repeated builds of the same project are faster.

2.3 Containers and Docker

A container runs a process in an isolated filesystem, network and process space using features of the Linux kernel. Docker [4] packages that into images and a command line. Two features matter here. Resource flags such as `--memory` and `--cpus` are enforced by control groups, and a process that exceeds its memory limit is killed by the kernel and reported as OOM killed. A user-defined bridge network lets containers reach each other by name, which Traefik uses to reach the production container.

2.4 Reverse proxies and service discovery

A reverse proxy receives requests and forwards them to the right backend, usually by hostname. With static configuration the file has to be edited and reloaded whenever a backend appears. Traefik [5] avoids this with providers. Its Docker provider watches the Docker event stream and reads labels on containers, so a new container with a router rule label gets a route as soon as it starts. Its file provider watches a directory of YAML files and reloads when one changes. Tugboat uses the Docker provider for branch previews and the file provider for the production route.

2.5 Tunnels

Cloudflare Tunnel [6] makes an outbound connection from the local machine to Cloudflare's network and forwards public requests back through it. The quick tunnel mode, started with `cloudflared tunnel --url`, needs no account. It prints a random address under `trycloudflare.com` that serves HTTPS. Because the connection is outbound, no router or firewall configuration is needed. The addresses are temporary and change every time a tunnel is started.

2.6 Blue-green deployment and rollback

In a blue-green deployment [7] the new version is started next to the running one and checked before any traffic is moved to it. Traffic is then switched in one step and the old version is kept or stopped. Because the previous version was working, a rollback is the same switch in the other direction. In Tugboat the two versions are two containers, and the switch is a change to Traefik's production route.

2.7 Webhooks and request signing

A webhook is an HTTP request that a service sends when something happens. GitHub sends a push event when commits are pushed and a delete event when a branch is removed. When a webhook has a secret, GitHub signs the raw request body with HMAC-SHA256 and sends the result in the `X-Hub-Signature-256` header [8]. The receiver recomputes the signature over the raw body with the same secret and compares the two in constant time. A request that fails the check is rejected. Tugboat gives every linked project its own random secret.

3. Technology stack

Figure 1 groups the tools by layer. The control plane is the only code I wrote. Every other box in the figure is an existing tool that the control plane starts and drives through its command line.

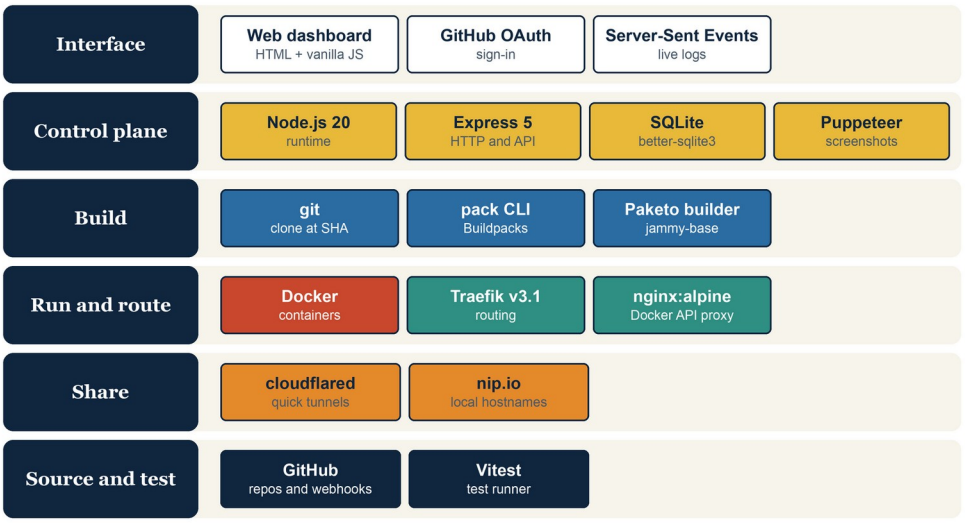


Figure 1: Technology stack by layer

3.1 Components

Table 1: Technology components and roles

Component	Version or source	Role in the project
Node.js	20 or newer	Runtime for the control plane
Express	5.2	HTTP server, REST API, static dashboard
better-sqlite3	12	SQLite storage for projects, deployments, sessions
Puppeteer	24 (headless Chromium)	Screenshot of each deployment
Docker (OrbStack)	current	Runs the built images as containers
pack and Paketo builder	builder-jammy-base	Builds images from source without a Dockerfile
Traefik	v3.1	Routes hostnames to containers
nginx	alpine image	Proxy in front of the Docker socket for Traefik
cloudflared	current	Quick tunnels for public HTTPS addresses
git	system	Clones the repository at the pushed commit
GitHub OAuth App and webhooks	GitHub API	Sign-in, repository access, push and delete events
Vitest	2	Unit and integration test runner

3.2 Dependencies

The control plane has three runtime dependencies. Everything else it needs is a command line program that it starts as a child process. The sample application used as a deployment target is a separate package with its own Express dependency.

Table 2: Node.js dependencies of the control plane

Package	Version	Purpose
express	^5.2.1	Web framework, routing, static files
better-sqlite3	^12.11.1	Synchronous SQLite driver
puppeteer	^24.43.1	Headless Chromium used for deployment screenshots
vitest (development)	^2.1.4	Test runner

4. System architecture

4.1 Overview

Figure 2 shows the architecture. GitHub sends push and delete events to the control plane. For each push the control plane clones the repository, builds an image with `pack`, and starts a container in Docker. Traefik is watching Docker and routes a hostname to the new container without being told. A `cloudflared` process, started by the control plane, forwards a public HTTPS address to the container's published port. The browser talks to the control plane over REST and receives build logs over Server-Sent Events.

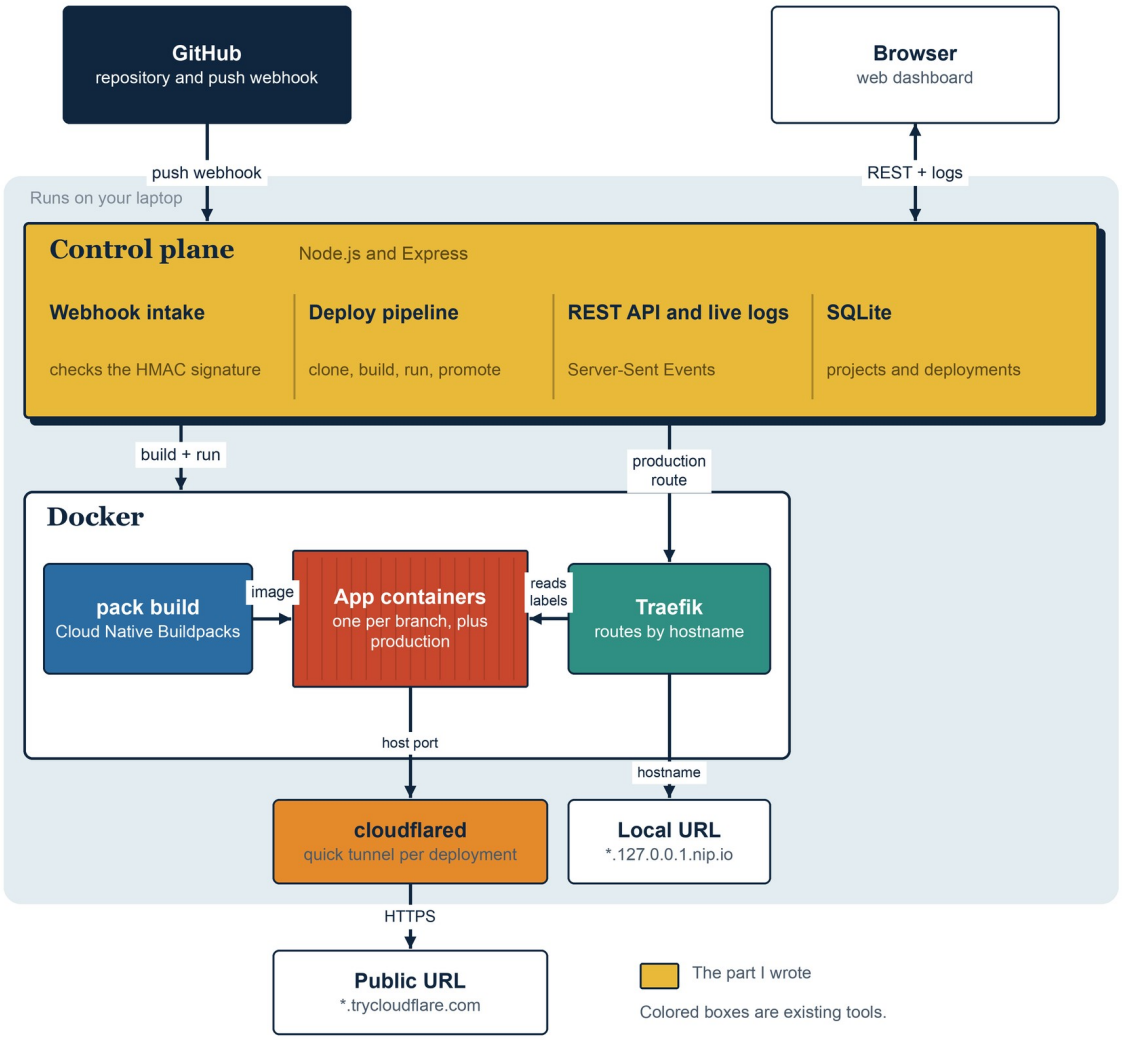


Figure 2: High-level system architecture

GitHub cannot reach a laptop directly, so at start-up the control plane opens its own `cloudflared` tunnel and registers webhooks that point at it. Every deployment can therefore be reached in two ways. The local URL, such as `feature-x.127.0.0.1.nip.io:8000`, goes through Traefik. The public URL goes through the tunnel straight to the container. Production has its own hostname, `production.127.0.0.1.nip.io`, which Traefik routes with a file-based rule.

4.2 Control plane modules

The control plane has 32 source files, each with one responsibility. Table 3 lists the main ones.

Table 3: Control plane modules

Module	Responsibility
server.js	Entry point. Starts Traefik and the control plane tunnel, serves the dashboard and REST API, runs the OAuth flow
projectPipeline.js	The deployment pipeline: lock, clone, build, run, tunnel, screenshot, promote
multiProjectWebhook.js	Routes a webhook to its project and verifies that project's own secret
hmac.js, pushEvent.js, deleteEvent.js	Signature check and parsing of GitHub payloads
cloneRepo.js	Clones a repository at an exact commit using the user's token
buildImage.js, buildWithLogging.js	Builds the pack command and streams its output to the database and the browser
runAppContainer.js, dockerLabels.js	Starts containers with Traefik labels, environment variables and limits
traefikController.js	Starts Traefik and the Docker API proxy on the shared network
productionRoute.js, promote.js	Writes the production route and performs the blue-green swap
rollback.js, deploymentHistory.js	Re-promotes the previous image from the promotion history
tunnelManager.js, cloudflaredUrl.js	Starts `cloudflared` and parses the assigned address
dnsReachability.js	Waits until a new tunnel name resolves on a public resolver
screenshot.js	Captures a deployment screenshot with headless Chromium
deployLock.js	Allows one deploy at a time for each repository and branch
previewRegistry.js, onDeleteTeardown.js	Tracks live previews and tears them down when a branch is deleted
repoCleanup.js	Removes everything a project owns when it is unlinked
resourceLimits.js	Turns memory and CPU settings into `docker run` flags
db.js	SQLite schema, migrations and queries
session.js, githubOAuth.js	Signed session cookies and the GitHub OAuth flow

4.3 Networking and ports

Table 4: Ports and networks

Port	Where	Purpose
4000	Host	Control plane: dashboard, REST API and webhook intake (`PORT`)
8000	Host, forwarded to Traefik port 80	Traefik web entry point for local URLs (`TRAEFIK_HTTP_PORT`)
8081	Host, forwarded to Traefik port 8080	Traefik API, used to confirm that a route has loaded (`TRAEFIK_API_PORT`)
2375	Docker network only	nginx proxy in front of the Docker socket
3000	Inside each container	The port an application must listen on
Random	Host	Published port of each container, the target of its tunnel

All containers join a Docker bridge network named `tugboat`. On this network Traefik reaches the production container by its container name, and the Docker API proxy is reachable at `tugboat-traefik-docker-proxy:2375`.

4.4 Repository structure

```
tugboat/
  sample-app/            minimal Express app used as a deployment target
  control-plane/
    src/                 one module per capability (32 files)
    public/index.html    dashboard (vanilla JavaScript, no build step)
    docker/              nginx config for the Docker API proxy
    tests/               15 unit, 12 integration and 1 smoke test file
    data/                SQLite file and screenshots (git-ignored)
  docs/                  README screenshots and the presentation
  README.md
```

5. Application design

5.1 Features

- Sign in with GitHub. Sessions survive a server restart.
- Link any repository the user can access, with an optional root directory for monorepos. Linking starts a first deployment.
- A preview deployment for every pushed branch, each with a local URL, a public URL and a screenshot.
- Production promotion when the default branch is pushed, and a one-click rollback.
- Redeploy of the current head of a branch without a new commit.
- Live build log streaming while `pack` runs.
- Per-project environment variables and memory and CPU limits.
- A history of the webhooks GitHub sent, with the outcome of each.
- Clean-up when a branch is deleted, and a complete clean-up when a project is unlinked.

5.2 REST API

Table 5 lists the routes. All routes except the webhook, the log stream and the screenshot require a signed-in session.

Table 5: Route summary

Method	Route	Behaviour
GET	/auth/login	Redirects to GitHub's authorize page
GET	/auth/callback	Exchanges the code for a token, creates a session
POST	/auth/logout	Destroys the session
GET	/api/me	Returns the signed-in user
GET	/api/repos	Lists the user's GitHub repositories
GET	/api/projects	Lists linked projects with their settings
POST	/api/projects	Links a repository: creates the project and webhook, starts the first deployment
PATCH	/api/projects/:owner/:name	Updates root directory, environment variables, limits
DELETE	/api/projects/:owner/:name	Unlinks the project and cleans up
POST	/api/projects/:owner/:name/deploy	Deploys the head of a branch
POST	/api/projects/:owner/:name/rollback	Rolls production back one promotion
GET	/api/projects/:owner/:name/webhook-deliveries	Recent webhook deliveries
GET	/api/deployments	Lists deployments, optionally for one repository

Method	Route	Behaviour
DELETE	/api/deployments/:id	Deletes a non-production deployment and its infrastructure
GET	/api/deployments/:id/stream	Build log as Server-Sent Events
GET	/api/deployments/:id/screenshot	Screenshot image of the deployment
POST	/webhook/github/:projectId	GitHub events, checked against the project's secret

5.3 Data model

Figure 3 shows the four SQLite tables. A deployment stores its whole build log in the `log` column and its state in `status`, which is one of `building`, `success`, `failed` or `torn_down`. Deployments are linked to projects by the repository name, not by a foreign key, so deployment history can be removed independently of the project row.

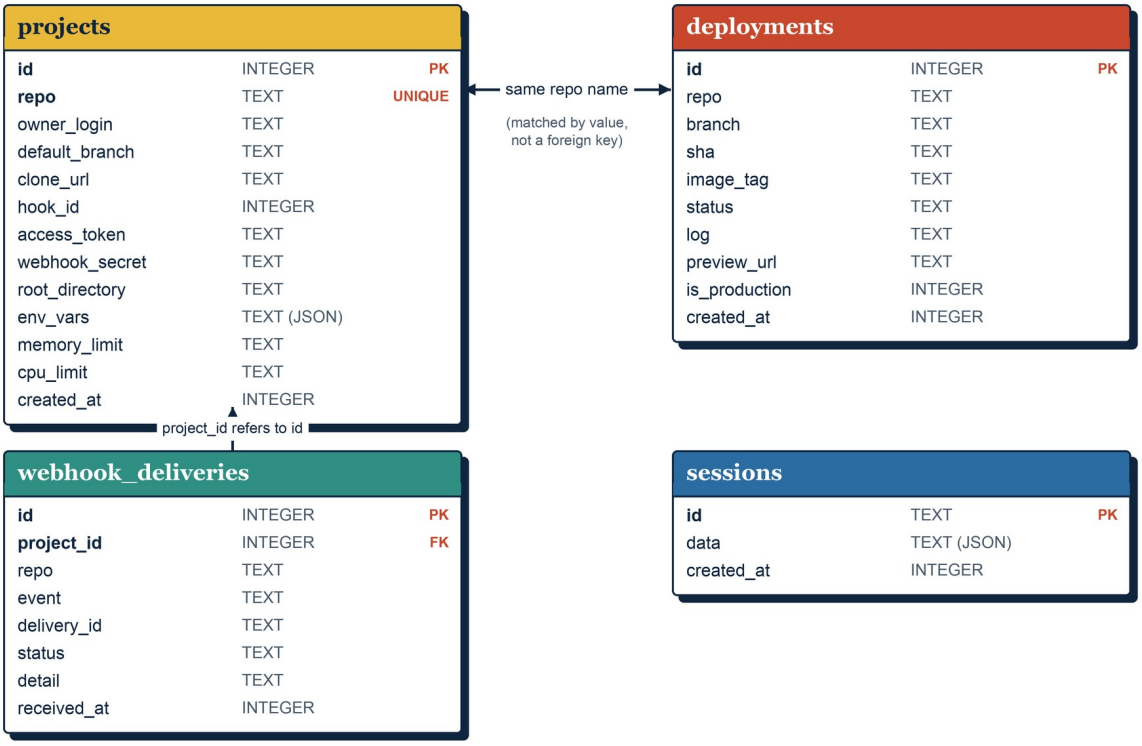


Figure 3: Database tables

The schema is created with `CREATE TABLE IF NOT EXISTS`. Columns added later are applied by `migrateColumns`, which reads `PRAGMA table_info` and runs `ALTER TABLE ... ADD COLUMN` for any column that is missing. An existing database file is therefore upgraded in place, and a test checks this against a real file.

5.4 Authentication and sessions

Sign-in uses a GitHub OAuth App with the `repo` scope. The login route generates a random state value and redirects to GitHub. The callback checks the state, exchanges the code for an access token, reads the user's profile and creates a session. The token is kept in the session and copied to a project when that project is linked, because webhooks arrive without a user.

A session is a random 24-byte identifier stored in the `sessions` table. The cookie value is the identifier followed by an HMAC-SHA256 of it, computed with `SESSION_SECRET`, and it is compared in constant time. The cookie is `httpOnly`, `sameSite=lax` and lasts 30 days. Because sessions live in SQLite and not in memory, restarting the server does not sign anyone out.

5.5 The web interface

The dashboard is a single HTML file with plain JavaScript and no build step. Figure 4 shows the landing page and a project page. The project grid shows the latest screenshot of each project. A project page lists its deployments, opens a deployment's build log inline, and has buttons for Rollback and Redeploy. Below the list are sections for the root directory, environment variables, resource limits, the unlink action and the webhook delivery history.

The page polls the API every four seconds. While a log panel is open the page is not redrawn, because redrawing replaced the element that the live stream was writing into (Chapter 11).

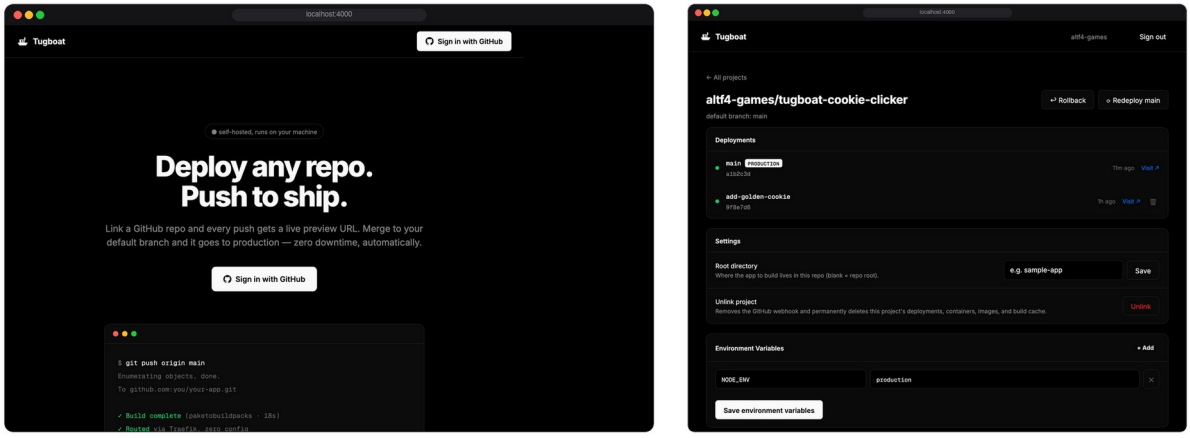


Figure 4: Landing page and project page (the project page is shown with sample data)

6. Deployment pipeline

Figure 5 shows the pipeline for one push. Steps 1 to 8 run for every push to any branch. Steps 9 and 10 run only when the branch is the project's default branch.

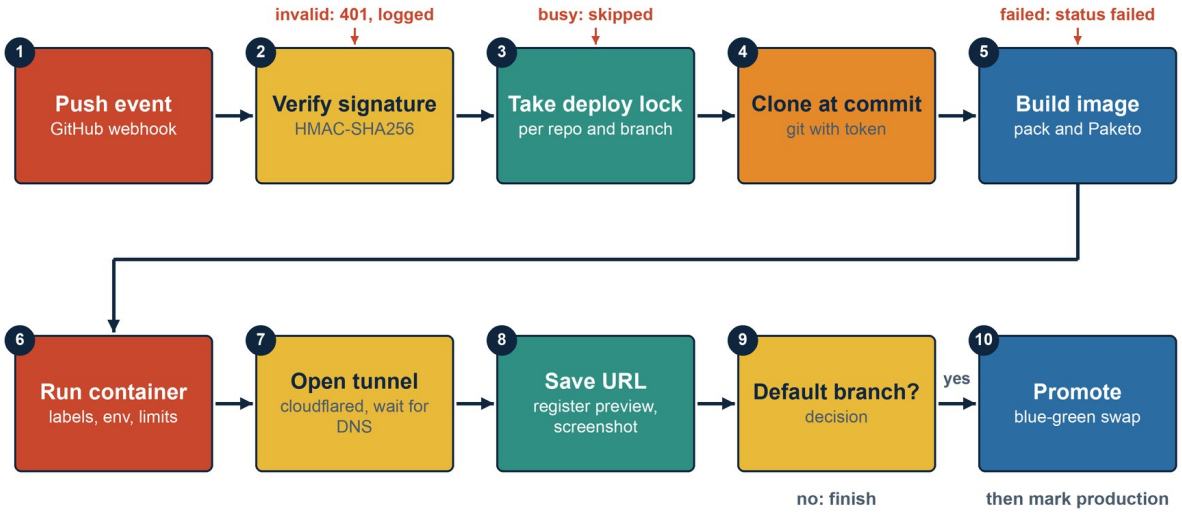


Figure 5: Deployment pipeline for one push

6.1 Push: webhook intake

Each project has its own webhook URL, `/webhook/github/<projectId>`, and its own secret of 32 random bytes. The route is mounted before `express.json()` so that the signature is computed over the exact bytes GitHub sent. The control plane looks up the project from the URL, so it knows which secret to check before it parses the body. A request with a missing or wrong signature gets a 401.

A push event gives the repository, the branch (from `ref`) and the commit SHA (from `after`). A delete event gives the branch and the reference type, and only branch deletions cause a teardown. Every delivery is recorded with one of four outcomes: `ok`, `ignored`, `invalid_signature` or `error`. The dashboard shows these, which makes a broken webhook easy to spot.

6.2 Build

The pipeline first takes a lock for the repository and branch (Section 6.8). It then clones the repository with `git clone --branch <branch> --single-branch` using the owner's token, and checks out the

exact commit. The directory is a temporary one and is removed when the run ends. If the project has a root directory, the build runs in that subfolder, which supports monorepos.

A deployment row is created with status `building`, and `pack` runs as a child process:

```
pack build tugboat/<owner>-<repo>:<sha> \
     --path <clone>/<root directory> \
     --builder paketobuildpacks/builder-jammy-base --trust-builder
```

The pipeline appends output from both streams to the deployment's `log` column and also emits it on an event emitter. The browser reads the emitter through the stream route. It first receives the log stored so far and then each new chunk, so a page opened in the middle of a build still shows the whole log. When `pack` exits with code 0 the status becomes `success` and the image tag is stored. Any other exit code gives `failed`.

6.3 Run and route

If the branch already has a live preview, its container and tunnel are stopped first. The new container is named `tugboat-preview-<id>` and started on the `tugboat` network with `PORT=3000`, the project's environment variables, any memory and CPU limits, a randomly chosen published port, and three labels:

```
traefik.enable=true
traefik.http.routers.<repo>-<branch>.rule=Host(`<branch>.127.0.0.1.nip.io`)
traefik.http.services.<repo>-<branch>.loadbalancer.server.port=3000
```

Traefik receives the container start event through the Docker API, reads the labels and creates a router and a service. Nothing is edited and nothing is reloaded. Traefik runs with `exposedbydefault=false`, so only labelled containers are routed. The `nip.io` service resolves any name of the form `<anything>.127.0.0.1.nip.io` to 127.0.0.1, so no hosts file is needed.

6.4 Share: tunnel, DNS and screenshot

The pipeline starts `cloudflared tunnel --url http://localhost:<port> --no-autoupdate` against the container's published port. It reads the combined output until it matches `https://<name>.trycloudflare.com`, which is the public address. A unit test checks this pattern against real captured `cloudflared` output.

`cloudflared` prints the address as soon as the tunnel registers, but the name can take several seconds to resolve. On my network the local resolver was slower than a public one, and links failed to open right after a deploy. The pipeline therefore waits, for up to 30 seconds, until the name resolves on the public resolver 1.1.1.1. If the wait times out, a warning is added to the log and the link is saved anyway.

The pipeline then saves the address on the deployment, registers the preview with its container name and tunnel process, and takes a screenshot in the background. Chromium resolves names through the operating system, which has the same delay, so the screenshot code resolves the address itself and passes it to Chromium with `--host-resolver-rules`. The page is loaded at 1280 by 800 pixels

and saved under `data/screenshots/<id>.png`. A failed screenshot is logged and never fails the deployment.

6.5 Promote: blue-green swap

When the pushed branch is the default branch, the image is promoted to production. Figure 6 shows the four steps.

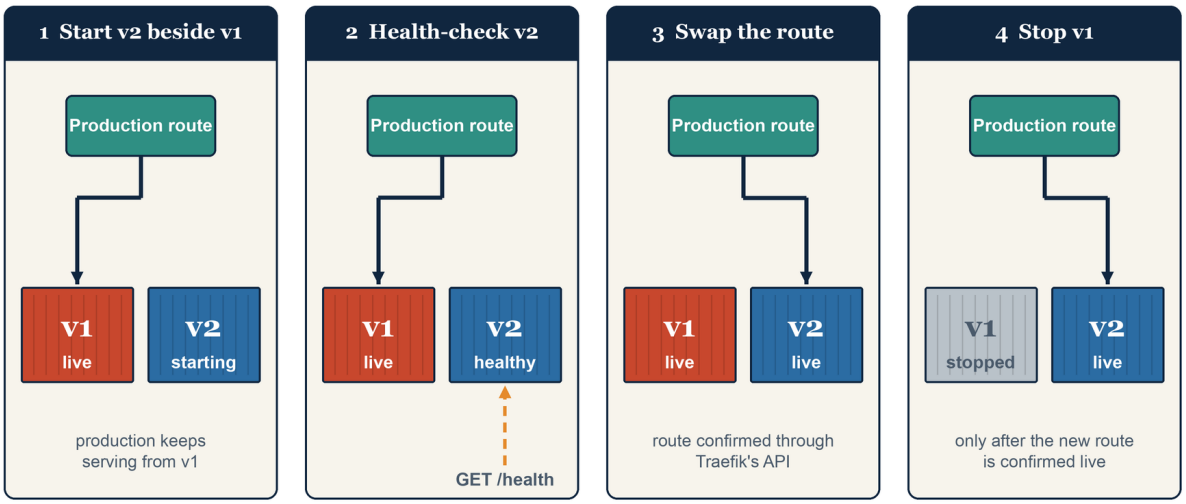


Figure 6: Blue-green promotion of a new image

1. The pipeline starts a second container, `tugboat-production-<repo>-<timestamp>`, with a published port. The old one keeps serving.
2. The control plane calls `GET /health` on the new container's published port, directly and not through Traefik, every 300 ms for up to 30 seconds, until it gets a 2xx response.
3. The control plane writes the production route file into the Traefik container with `docker cp`, then polls Traefik's API until the `production@file` service points at the new container, every 200 ms for up to 15 seconds.
4. The control plane records the promotion in the history, then stops and removes the previous container.

The route file is small. It has one router, one service and one server URL:

```
http:
  routers:
    production:
      rule: "Host(`production.127.0.0.1.nip.io`)"
      service: production
      entryPoints: [web]
  services:
    production:
      loadBalancer:
        servers:
          - url: "http://<container name>:3000"
```

I first used Docker labels for production too, with two containers behind one router. Traefik sent traffic to the new container before its first health check had passed, and requests were dropped. With the file provider there is exactly one backend in the route at any time, and the route changes in one step after the health check. If the health check or the route check fails, an error is added to the log and the old container keeps running, because it is stopped last.

6.6 Rollback

The control plane keeps a list of promotions for each repository, with the image tag and container of each. A rollback needs at least two entries. It calls the same promote function with the previous image tag and the current container as the one to stop. A rollback is recorded as a new promotion, so a second rollback returns to the version that was just replaced. The dashboard then marks the deployment that owns the restored image as production. The list is kept in memory (Section 12.2).

6.7 Branch deletion and unlinking

When GitHub reports that a branch was deleted, Tugboat stops the preview container and its tunnel, removes the registry entry, and marks the deployment `torn_down` so that the dashboard no longer shows a dead link. You can also delete a deployment from the dashboard. If it is the live preview, its container and tunnel are stopped first. The current production deployment cannot be deleted.

Unlinking a project removes everything it owns, so that disk use does not grow with every repository that is tried:

- the GitHub webhook, deleted through the GitHub API;
- all preview containers and tunnels, and the production container, with the production route cleared if this project held it;
- the deployment rows, webhook deliveries and screenshots in the database and on disk;
- every image tagged `tugboat/<repo>` and the `pack-cache-tugboat_<repo>_*` build cache volumes.

Preview containers have predictable names, `tugboat-preview-<id>`, so the clean-up looks them up by name as well as through the in-memory registry. The registry is empty after a restart, and relying on it alone left containers behind (Chapter 11).

6.8 Concurrency control

Two overlapping runs for the same branch used to race on the single preview slot: the second run stopped the first run's container and tunnel while it was still the live preview. A deploy lock now serialises runs for each repository and branch. It is a set of keys in memory, and taking the lock is one synchronous check-and-add, so two requests arriving back to back cannot both get it. A request that finds the lock held is skipped. Different branches of the same repository do not block each other.

Table 6 summarises what happens when each stage fails.

Table 6: What happens when a stage fails

Stage	Behaviour on failure
Signature check	401 is returned, the delivery is recorded as `invalid_signature`, and nothing starts
Deploy lock	A second request for the same repository and branch is skipped while one is running
Clone or build	The deployment is marked `failed` and the output so far stays in its log
Container start or tunnel	An error is added to the log. The branch's previous preview was already stopped
DNS wait	A warning is added to the log and the link is saved anyway
Screenshot	Logged to the server console only. The deployment is not affected
Promote	An error is added to the log and the previous production container keeps running

7. Containerisation and runtime controls

7.1 Image build

Images are built by `pack` with the Paketo builder, which produces `linux/amd64` images. On Apple silicon they run under emulation, and Docker prints a platform warning each time. Each image is tagged with the repository and the commit, so a given commit always maps to one tag. `pack` keeps two cache volumes for each image tag, named `pack-cache-tugboat_<repo>_<sha>-<hash>.build` and `.launch`. Together with the image itself they are what a build leaves behind, which is why unlinking removes both.

7.2 Container configuration

Table 7: How each container is configured

Setting	How it is applied
Port	`PORT=3000` in the environment and `-p 0:3000`, which publishes a random host port
Network	`--network tugboat`
Environment variables	One `-e KEY=VALUE` for each variable saved on the project
Memory and CPU	`--memory <value>` and `--cpus <value>` when the project has limits
Routing labels	The three Traefik labels (previews only)
Names	`tugboat-preview-<id>` and `tugboat-production-<repo>-<timestamp>`

Environment variables and limits are read when a container starts, so a change applies to the next deployment and not to containers that are already running.

7.3 Traefik and the Docker API proxy

Table 8: Traefik start-up flags

Flag	Purpose
`--providers.docker=true`	Discover containers through the Docker API
`--providers.docker.exposedbydefault=false`	Route only containers that have `traefik.enable=true`
`--providers.docker.network=tugboat`	Use the container's address on the shared network
`--providers.docker.endpoint=tcp://tugboat-traefik-docker-proxy:2375`	Reach Docker through the nginx proxy, not the raw socket
`--entrypoints.web.address=:80`	The HTTP entry point, published on host port 8000
`--providers.file.directory=/etc/traefik/dynamic`, `--providers.file.watch=true`	Load and watch the production route file
`--api.insecure=true`	Expose the API on port 8080, used to confirm that a route loaded

Traefik's Docker client sends requests for API version 1.24 and never negotiates a newer one. OrbStack's daemon rejects anything below 1.40. The small nginx proxy accepts Traefik's requests, strips the `/vX.Y/` prefix from the path and forwards them to the real socket, so the daemon answers with its current version. Response buffering is turned off, because Traefik relies on the long-lived `/events` stream to hear about containers starting and stopping.

7.4 Resource limits

A project can set a memory limit such as `512m` and a CPU limit such as `1.0`. These become `--memory` and `--cpus` on both preview and production containers. The limit is enforced by the kernel. A test starts a container with a 20 MB limit that allocates 300 MB and checks through `docker inspect` that Docker killed it (`OOMKilled` is true and the exit code is not zero). A second test runs a program that allocates 50 MB under a 512 MB limit and checks that it exits normally.

8. Testing

8.1 Approach

Each feature began as a failing test, then I wrote the code to make it pass. Two rules shaped the tests. First, integration tests run the real tools: a real `pack build`, a real Docker daemon, a real Traefik container and a real `cloudflared` tunnel. Second, no data is invented. The webhook tests use payloads captured from real GitHub deliveries, and the database only ever holds rows created by real builds and real webhooks.

Unit tests cover logic with no outside effects, such as signature verification, label construction and the address pattern. Integration tests are slower, and they show whether the system works against the real tools. Vitest runs test files one after another, because several integration tests share one GitHub repository and would otherwise receive each other's webhook events.

8.2 Test suite

Table 9: Test suite overview

Area	What is checked
Webhook intake	Signature accepted, wrong secret rejected, tampered body rejected, missing header rejected; real push and delete payloads parsed; routing by project and unknown project gives 404; a real webhook delivery from GitHub
Build	Command construction and tag format; a real `pack build` produces a runnable image
Routing	Label and hostname construction; Traefik routes a first container, then a second one with no restart; production route file is valid YAML
Sharing	Address pattern against real `cloudflared` output; a real tunnel reaches the local app over HTTPS
Promote and rollback	Two promotions with zero failed requests; rollback restores the first image; history order
Concurrency	Lock acquire, release and per-branch independence; two overlapping real deploys do not race
Teardown	Deleting a real branch removes the container, the route and the tunnel process
Resource limits	Flag construction; a real container over its limit is OOM killed; one under it is not
Persistence and sign-in	Deployments and projects stored and queried in real SQLite; schema migration of an existing file; sessions survive reopening the database; tampered or wrongly signed cookies rejected
Dashboard and clone	Live build output reaches the stream as it is produced; a private repository is cloned at an exact commit

8.3 Results

The suite has 74 tests in 28 files: 15 unit test files, 12 integration test files and one smoke test that builds the sample application. The last complete run took about 411 seconds, nearly all of it real builds and container starts.

- A poller requests the production URL every 50 ms while production is promoted twice in a row. The test requires more than ten requests, zero failures, and that the old container is gone afterwards.
- After two promotions and one rollback, the running container's image equals the first image, `/health` returns 200, and the replaced container has been removed.
- A container over its memory limit is killed by Docker and reports `OOMKilled`. A container well under the limit is not.
- After a real branch is pushed and then deleted, the container, the Traefik route and the tunnel process are all gone.
- The sample application builds into a runnable image in about 21 seconds when the base layers are cached.

> These figures come from the last complete run, which was before the final dashboard, clean-up and screenshot changes. Those changes were checked one by one against real Docker, Traefik and `cloudflared`, but I did not re-run the whole suite afterwards.



9. Environment configuration

9.1 Prerequisites

I developed and tested Tugboat on macOS with an Apple silicon processor and OrbStack as the Docker runtime. Table 10 lists what must be installed.

Table 10: Prerequisites

Tool	Needed for	Install
Docker or OrbStack	Running every container; must be running before the server starts	`brew install --cask docker` or OrbStack
pack	Building images	`brew install buildpacks/tap/pack`
cloudflared	Tunnels	`brew install cloudflare/cloudflare/cloudflared`
Node.js 20 or newer	The control plane	`brew install node`
git	Cloning repositories	Included with macOS developer tools
GitHub OAuth App	Sign-in	Created once in GitHub developer settings

9.2 Configuration

The GitHub OAuth App uses `http://localhost:4000` as its home page and `http://localhost:4000/auth/callback` as its callback URL. Its credentials go into `control-plane/.env`, which is git-ignored.

Table 11: Configuration variables

Variable	Required	Default	Purpose
GITHUB_CLIENT_ID	Yes	none	OAuth App client id
GITHUB_CLIENT_SECRET	Yes	none	OAuth App client secret
SESSION_SECRET	Advised	random each start	Key that signs session cookies; set it so sessions survive restarts
PORT	No	4000	Control plane port
TRAEFIK_HTTP_PORT	No	8000	Host port for local URLs
TRAEFIK_API_PORT	No	8081	Host port for Traefik's API

Per-project settings (root directory, environment variables, memory limit, CPU limit) are edited in the dashboard and stored in the `projects` table.

9.3 Running and cleaning up

```
cd control-plane
npm install
cp .env.example .env      # fill in the three values
npm start                 # open http://localhost:4000
```

The first build downloads the Paketo builder and run images, about 5 GB in total, so do one build before a demo. Unlinking a project cleans up that project. To remove everything Tugboat has created, for example after a demonstration:

```
docker ps -a --filter "name=tugboat" --format "{{.Names}}" | xargs -r docker rm -f
docker rmi -f $(docker images --filter "reference=tugboat/*" -q)
docker volume prune -a -f
```

10. End-to-end workflow

Figure 7 follows one push through every component, in order. Steps 11 to 14 happen only for the default branch.

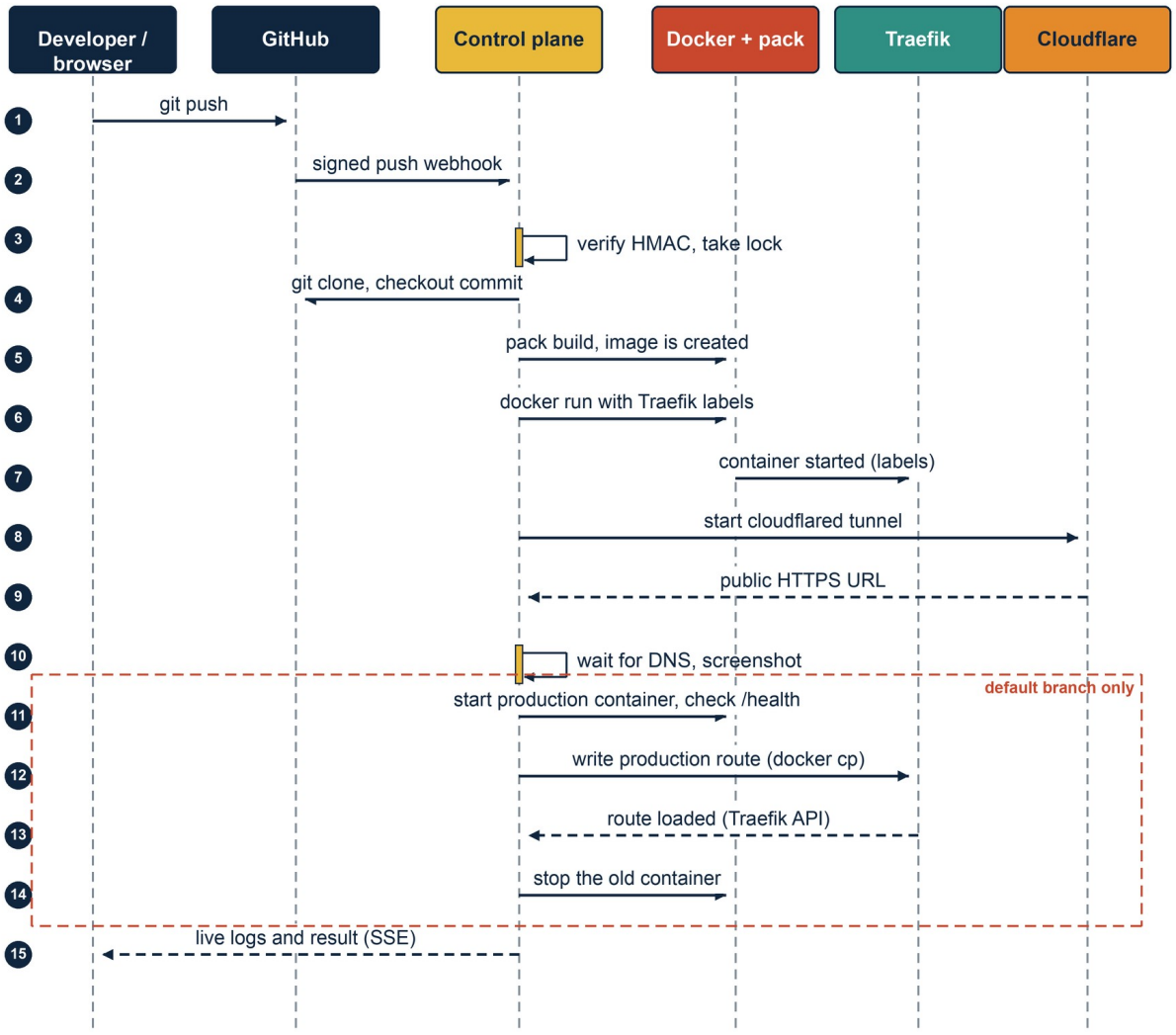


Figure 7: End-to-end sequence for one push

To check the whole system I used a second repository, `tugboat-cookie-clicker`. It is a React and Vite game with a small Express server that serves the built files and answers `/health`. The run went as follows:

1. Start the control plane. It starts Traefik and its own tunnel, then prints the webhook address.
2. Sign in with GitHub and link the repository. Tugboat creates the webhook and starts the first deployment.
3. Open the deployment in the dashboard and watch the `pack` output stream.
4. Open the preview address. It is a public HTTPS link, and a screenshot appears on the project card.
5. Push a commit to the default branch. A new preview is built, and production swaps to the new image.
6. Press Rollback. Production returns to the previous image.
7. Unlink the project. The webhook, containers, images and build cache are removed.

11. Problems encountered and solutions

Most of these appeared only because the tests ran against real tools. A mock would have passed in every case.

Table 12: Problems and solutions

Problem	Cause	Solution
Traefik could not use Docker on OrbStack	Its client sends API 1.24 and OrbStack requires 1.40 or newer	An nginx proxy strips the version prefix; buffering is off so `/events` streams
Requests dropped during a production swap	Label discovery routed to the new container before its first health check	Use the file provider and swap one backend in one step after the health check
Route file changes were not noticed	Writes through a bind mount do not trigger inotify across the macOS VM	Write the file with `docker cp`, which is a native write inside the container
Integration tests disturbed each other	Several files shared one GitHub webhook and received each other's events	Run test files one at a time and filter events by branch
The memory limit test passed when it should not	`Buffer.alloc` zero-fills without committing the pages	Write non-zero bytes and touch every page
Double-clicking Redeploy killed a live preview	Two runs for one branch raced on the preview slot	A synchronous deploy lock for each repository and branch
Preview links failed right after a deploy, and screenshots failed	New tunnel names resolve slowly on the local resolver	Wait on a public resolver, then give Chromium the address directly
The sample React app crashed in production	`vite preview` needs a development dependency that the build removes	Serve the built files with a small Express server that also answers `/health`
The test app failed to build	A lockfile made on macOS lacked the Linux native rollup binary	Regenerate the lockfile inside a `linux/amd64` container
The log viewer froze on "connecting"	The four-second redraw replaced the element the live stream wrote into	Skip the redraw while a log panel is open
Unlinking left containers behind	Clean-up relied on an in-memory registry that is empty after a restart	Also stop containers by their predictable names

12. Conclusion and future work

12.1 Conclusion

Tugboat shows that the push-to-deploy workflow can be rebuilt from open-source parts and run on one machine. A linked repository is built without a Dockerfile, each branch gets a routed container and a public HTTPS link, the default branch is promoted without dropping requests, and a rollback returns to the previous image. I met each objective in Section 1.3. The pipeline, routing, promotion, rollback and teardown are covered by tests against the real tools, and I checked the dashboard by hand.

Most of a deployment platform is coordination between tools that each work well on their own. The hard problems came up where two tools met: an API version that one tool sends and another rejects, a file write that one system sees and another does not, a name that one resolver knows and another does not yet. Testing against the real tools exposed them.

12.2 Limitations

- There is one production route, so only one project can be in production at a time.
- The promotion history is kept in memory, so a restart clears it and a rollback needs two new promotions first.
- A deployment's status reflects its build. A later failure, such as in the tunnel or the promotion, appears in the log but the status still reads `success`.
- Quick tunnel addresses change with every deployment and need Cloudflare to be reachable.
- OAuth tokens, webhook secrets and environment variables are stored unencrypted in SQLite.
- The tool is single-tenant. Any signed-in user can manage every project, and the log stream and screenshot routes do not check the session.
- An application must read `PORT` and answer `/health` to be promoted. Only runtime variables are supported.

12.3 Future work

- Give each project its own production route and hostname, and store the promotion history in SQLite so rollback survives a restart.
- Use named Cloudflare tunnels with custom domains for stable addresses.
- Encrypt tokens and variables at rest, and limit each project to the user who linked it.
- Track a deployment's whole life cycle in its status, and add build-time variables and previews for pull requests.

References

[1] A. Wiggins, "The Twelve-Factor App," https://12factor.net/.

[2] Cloud Native Buildpacks, "Buildpacks documentation and specification," https://buildpacks.io/docs/.

[3] Paketo Buildpacks, "Paketo Buildpacks documentation," https://paketo.io/.

[4] Docker, Inc., "Docker documentation," https://docs.docker.com/.

[5] Traefik Labs, "Traefik documentation," https://doc.traefik.io/traefik/.

[6] Cloudflare, "Cloudflare Tunnel documentation," https://developers.cloudflare.com/cloudflare-one/.

[7] M. Fowler, "BlueGreenDeployment," https://martinfowler.com/bliki/BlueGreenDeployment.html.

[8] GitHub, "Webhooks documentation," https://docs.github.com/en/webhooks.

[9] GitHub, "Authorizing OAuth apps," https://docs.github.com/en/apps/oauth-apps.

[10] The Express team, "Express documentation," https://expressjs.com/.

[11] Vitest, "Vitest documentation," https://vitest.dev/.

[12] Puppeteer, "Puppeteer documentation," https://pptr.dev/.

[13] SQLite, "SQLite documentation," https://www.sqlite.org/docs.html.